Projeto final — Introdução ao Quarkus

Trabalho em equipe · enunciado para os alunos · conteúdo avaliado: Aulas 01 a 06

> **Tema livre:** cada equipe escolhe o domínio da própria API. Serão avaliados os conhecimentos técnicos demonstrados, não a complexidade ou originalidade do tema.

1. Objetivo

Desenvolver **uma API REST em Quarkus** que resolva um pequeno problema de negócio real ou simulado. A API deve persistir dados, consumir um serviço HTTP, disponibilizar health checks e métricas, lidar com falhas e validar requisições.

O projeto `catalogo-api`, desenvolvido nas Aulas 03–06, é uma **referência de implementação**. A equipe pode escolher outro domínio ou reutilizar a ideia com implementação própria.

Temas possíveis

Tema	Recurso persistente	Integração HTTP possível	Regra de negócio simples
Biblioteca	Livros / empréstimos	Catálogo externo ou mock	Impedir empréstimo de exemplar indisponível
Agendamentos	Agendamentos	Calendário de feriados ou mock	Evitar horários sobrepostos
Controle de tarefas	Tarefas	Serviço externo de prioridades ou mock	Impedir conclusão de tarefa cancelada
Gestão de eventos	Eventos / inscrições	Serviço de endereços ou mock	Limitar inscrições às vagas disponíveis
Gestão de filmes	Filmes favoritos	Catálogo externo ou mock	Impedir cadastro duplicado
Controle de despesas	Despesas	Serviço de câmbio ou mock	Impedir despesas com valor negativo

É permitido usar um serviço HTTP **simulado localmente** para facilitar a execução e a demonstração.

2. Escopo mínimo

- Uma aplicação **Java 21 + Quarkus + Maven**, com API REST em JSON e sem necessidade de frontend.
- Um recurso principal com **CRUD completo**, contendo pelo menos três propriedades úteis além do identificador.

- Uma regra de negócio em serviço gerenciado pelo CDI, distinta da simples delegação ao repositório.
- Um REST Client para consumir outro serviço HTTP ou mock local.
- Persistência com Hibernate ORM e Panache em um **banco relacional**: H2, PostgreSQL ou outro suportado pelo Quarkus. **PostgreSQL e Docker Compose não são obrigatórios nem dão ponto extra.**
- Configuração por profile, liveness, readiness, resiliência, métrica de negócio e validação/tratamento de erros.
- `README.md` e coleção Postman (ou comandos HTTP equivalentes) para reproduzir as demonstrações.

O uso correto de **Maven e JSON/DTOs** faz parte da execução normal do projeto, mas **não tem pontos separados**. Não há requisito de autenticação ou autorização nesta avaliação.

3. Organização sugerida do projeto

Recomendamos **Package by Feature + Boundary / Domain / Repository**, como na Aula 05:

```
src/main/java/br/com/equipe/projeto/
├── tarefas/                    # Exemplo; escolha o nome da sua funcionalidade
│   ├── boundary/               # Resource e DTOs HTTP
│   ├── domain/                 # Service e regras de negócio
│   └── repository/             # Entity e PanacheRepository
├── integracao/
│   └── servicoexterno/         # REST Client e contratos
└── platform/
    ├── health/                 # Liveness e readiness
    └── metrics/                # Opcional: métricas em componente próprio
```

O `Resource` trata a interface HTTP; o `Service`, a regra de negócio; o `Repository`, a persistência. Uma organização equivalente é aceita desde que seja clara e coerente.

4. Pontuação e correção

- São **16 requisitos, de 1 ponto cada**, com pontuação binária: **0 ou 1**.
- O requisito precisa estar **implementado e demonstrável** (código mais comportamento ou teste reproduzível).
- **Nota:** `(pontos obtidos ÷ 16) × 10`, arredondada para uma casa decimal.
- Não há ponto extra por tema escolhido, quantidade de endpoints além do necessário ou uso de PostgreSQL/Docker.

5. Requisitos avaliados (1 ponto cada)

Aulas 01 e 02 — HTTP, REST e Lombok (3 pontos)

Requisito 01 — Recursos e métodos HTTP (1 ponto)

Requisito: Criar recursos com URIs claras e utilizar GET, POST, PUT e DELETE conforme o significado da operação.

Critério de aceitação: Demonstrar ao menos um recurso, com GET de coleção, GET por ID, POST, PUT e DELETE, sem verbos no caminho (ex.: `/tarefas`, `/tarefas/{id}`).

Evidência esperada: Rotas e requisições Postman executadas.

Requisito 02 — Códigos de status e contratos (1 ponto)

Requisito: Retornar códigos de status HTTP coerentes com os cenários da API.

Critério de aceitação: Demonstrar 200 (consulta), 201 (criação), 204 (exclusão), 400 (requisição inválida) e 404 (recurso ausente); informar `Location` ao criar, quando aplicável.

Evidência esperada: Postman com respostas e status verificáveis.

Requisito 03 — Lombok aplicado conscientemente (1 ponto)

Requisito: Usar Lombok para evitar código repetitivo em pelo menos uma classe apropriada.

Critério de aceitação: Aplicar, por exemplo, `@Getter`, `@Setter` ou `@Builder`, explicar o código gerado e evitar `@Data` indiscriminado em entidade JPA.

Evidência esperada: Classe anotada e uso compilável.

Aula 03 — Quarkus REST, CDI e REST Client (3 pontos)

Requisito 04 — API Quarkus/JAX-RS (1 ponto)

Requisito: Implementar endpoints com Quarkus REST e anotações Jakarta REST.

Critério de aceitação: Demonstrar `@Path`, `@GET` ou `@POST`, `@Produces`/`@Consumes` quando necessários e retorno JSON correto.

Evidência esperada: Classe `*Resource` e resposta real.

Requisito 05 — CDI e regra de negócio (1 ponto)

Requisito: Injetar um serviço via CDI e implementar uma regra de negócio no serviço.

Critério de aceitação: O Resource delega a um serviço CDI injetado (por exemplo, com `@ApplicationScoped`); o serviço executa uma regra de negócio que vai além de apenas repassar a chamada ao repositório.

Evidência esperada: Resource + service + cenário da regra.

Requisito 06 — Consumo de API externa (1 ponto)

Requisito: Consumir outro serviço HTTP com Quarkus REST Client.

Critério de aceitação: Usar interface cliente e configuração de URL; demonstrar resposta do serviço externo ou mock local controlado.

Evidência esperada: Client + endpoint integrado + resposta.

Aula 04 — MicroProfile, observabilidade e resiliência (4 pontos)

Requisito 07 — Configurações e profiles (1 ponto)

Requisito: Externalizar ao menos uma configuração real e demonstrar dois profiles.

Critério de aceitação: Usar MicroProfile Config ou `@ConfigProperty`, `dev` e `prod`/`staging`, com ao menos uma variável de ambiente sobrescrevendo valor.

Evidência esperada: Arquivo(s) properties + execução em dois profiles.

Requisito 08 — Liveness e readiness (1 ponto)

Requisito: Disponibilizar duas verificações de saúde com propósitos distintos.

Critério de aceitação: `/q/health/live` e `/q/health/ready` respondem e a equipe explica quando cada uma seria DOWN; readiness tem condição controlável.

Evidência esperada: Health checks + requisições de teste.

Requisito 09 — Tolerância a falhas (1 ponto)

Requisito: Tratar falha de integração externa com mecanismo de resiliência.

Critério de aceitação: Implementar `@Retry` e `@Fallback` (ou equivalente MicroProfile Fault Tolerance), simular indisponibilidade e demonstrar resposta controlada.

Evidência esperada: Cenários externo UP/DOWN e classe responsável.

Requisito 10 — Métrica de negócio (1 ponto)

Requisito: Registrar uma métrica de negócio com Micrometer.

Critério de aceitação: Implementar pelo menos um contador ou timer e demonstrar seu valor em `/q/metrics`, variando após requisições.

Evidência esperada: Métrica Prometheus antes/depois.

Aula 05 — Hibernate ORM e Panache (3 pontos)

Requisito 11 — Entidade JPA (1 ponto)

Requisito: Mapear uma entidade persistente com Hibernate ORM.

Critério de aceitação: Ter `@Entity`, identificador e campos adequados ao domínio; não usar diretamente o DTO HTTP como entidade.

Evidência esperada: Classe Entity + tabela criada.

Requisito 12 — CRUD Panache Repository (1 ponto)

Requisito: Persistir e consultar dados utilizando Panache.

Critério de aceitação: Utilizar `PanacheRepository` (ou `PanacheEntity`, se justificado) e realizar CRUD completo sobre o recurso principal.

Evidência esperada: Código de persistência + CRUD real.

Requisito 13 — Transações e atualização (1 ponto)

Requisito: Garantir consistência nas operações de escrita.

Critério de aceitação: Operações de criação/atualização/exclusão usam `@Transactional` quando necessário; demonstrar que PUT altera o dado consultado depois.

Evidência esperada: Serviço transacional + sequência PUT/GET.

Aula 06 — Validação e tratamento de erros (3 pontos)

Requisito 14 — Bean Validation (1 ponto)

Requisito: Validar dados de entrada por anotações.

Critério de aceitação: Utilizar `@Valid` com pelo menos duas restrições distintas, como `@NotBlank`, `@Size`, `@Min` ou `@Positive`.

Evidência esperada: Requisição inválida rejeitada com 400.

Requisito 15 — ExceptionMapper personalizado (1 ponto)

Requisito: Tratar exceção de negócio com ExceptionMapper.

Critério de aceitação: Definir ao menos uma exceção específica (ex.: conflito de estado) e mapear para status apropriado sem `try/catch` repetido em cada endpoint.

Evidência esperada: Exceção, mapper e cenário HTTP.

Requisito 16 — Resposta de erro padronizada (1 ponto)

Requisito: Padronizar erros REST e validar os cenários.

Critério de aceitação: Responder JSON consistente com, por exemplo, `codigo`, `mensagem` e `timestamp`, em ao menos dois erros distintos (ex.: 400 e 404).

Evidência esperada: Postman mostrando corpos de erro e dois testes.

6. Entrega

Enviar o link do **repositório Git** (ou ZIP, caso a plataforma do curso exija) com:

```
nome-da-api/
├── pom.xml
├── README.md
├── src/
│   ├── main/
│   └── test/
├── docs/
│   └── evidencias.md            # Evidências dos requisitos 01 a 16
├── postman/
│   └── projeto-final.postman_collection.json
├── docker-compose.yml           # Opcional
└── .gitignore
```

No README, incluir **nome e integrantes da equipe**, problema resolvido, versões de Java e Quarkus, instruções para iniciar a API e eventual serviço externo/mock, banco escolhido, configurações e profiles, exemplos de requisições e uma tabela que indique onde encontrar a evidência de cada requisito.

Roteiro sugerido para a demonstração

1. Apresentar o domínio escolhido e mostrar as camadas `Resource → Service → Repository`.
2. Demonstrar CRUD com respostas HTTP corretas e uma regra de negócio executada pelo CDI.
3. Mostrar consumo do serviço externo, desligá-lo e testar `@Retry` / `@Fallback`.
4. Demonstrar configuração por profile, `/q/health/live`, `/q/health/ready` e `/q/metrics`.
5. Enviar entradas inválidas e apresentar o `ExceptionMapper` e as respostas JSON padronizadas.

7. Condições de avaliação

- Todos os temas são corrigidos pelos **mesmos 16 requisitos**.
- Mock local pode substituir uma API pública, desde que a interação HTTP seja real e reproduzível.
- O banco deve ser relacional e funcionar para o CRUD; **H2 é suficiente**.